— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 4

8 Semantics of Programming Languages (nk480)

Operational Semantics

(*a*) Suppose we have a language with booleans, integers, and *mutable* variables:

$$\begin{array}{rcl} e & ::= & n \mid e_0 + e_1 \mid e_0 < e_1 \mid \mathsf{true} \mid \mathsf{false} \mid \mathsf{if}\ e\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2 \\ & \mid & x \mid \mathsf{var}\ x = e_0\ \mathsf{in}\ e_1 \mid x := e \end{array}$$

(*i*) Give a grammar for the *values* of this language. [1 mark]

(*ii*) What mathematical object should be used to represent a store σ (which tracks which values each variable has)? [1 mark]

(*iii*) Give a reasonable operational semantics for this language, as a transition relation. (You may assume the existence of a substitution operation $\{v/x\}e$.)

$$\langle \sigma; e \rangle \rightsquigarrow \langle \sigma'; e' \rangle$$

This semantics should ensure (though you need not prove) that for any configuration $\langle \sigma; e \rangle$, it is either of the form $\langle \sigma; v \rangle$ with no further transitions, or otherwise it has at most one transition $\langle \sigma; e \rangle \rightsquigarrow \langle \sigma'; e' \rangle$. In addition to the formal rules, give an explanation of the reduction rules you define for variable declarations $\mathsf{var}\ x = e_0\ \mathsf{in}\ e_1$ and assignments $x := e$.

[8 marks]

Answer:

(*i*) The values are:

$$v ::= n \mid \mathsf{true} \mid \mathsf{false}$$

(*ii*) The environment σ is a partial function from variables to values.

(*iii*) A left-to-right, call-by-value semantics is:

$$\frac{m = \text{the sum of } n \text{ and } n'}{\langle \sigma; n + n' \rangle \rightsquigarrow \langle \sigma; m \rangle} \qquad \frac{m = \begin{cases} \mathsf{true} & \text{when } n \text{ less than } n' \\ \mathsf{false} & \text{otherwise} \end{cases}}{\langle \sigma; n < n' \rangle \rightsquigarrow \langle \sigma; m \rangle}$$

$$\frac{\langle \sigma; e_0 \rangle \rightsquigarrow \langle \sigma'; e_0' \rangle}{\langle \sigma; e_0 + e_1 \rangle \rightsquigarrow \langle \sigma'; e_0' + e_1 \rangle} \qquad \frac{\langle \sigma; e_1 \rangle \rightsquigarrow \langle \sigma'; e_1' \rangle}{\langle \sigma; v_0 + e_1 \rangle \rightsquigarrow \langle \sigma'; v_0 + e_1' \rangle}$$

$$\frac{\langle \sigma; e_0 \rangle \rightsquigarrow \langle \sigma'; e_0' \rangle}{\langle \sigma; e_0 \times e_1 \rangle \rightsquigarrow \langle \sigma'; e_0' \times e_1 \rangle} \qquad \frac{\langle \sigma; e_1 \rangle \rightsquigarrow \langle \sigma'; e_1' \rangle}{\langle \sigma; v_0 \times e_1 \rangle \rightsquigarrow \langle \sigma'; v_0 \times e_1' \rangle}$$

$$\frac{\sigma(x) = v}{\langle \sigma; x \rangle \rightsquigarrow \langle \sigma; v \rangle}$$

$$\frac{\langle \sigma; e \rangle \rightsquigarrow \langle \sigma'; e' \rangle}{\langle \sigma; x := e \rangle \rightsquigarrow \langle \sigma'; x := e' \rangle} \qquad \frac{}{\langle [\sigma \mid x : v']; x := v \rangle \rightsquigarrow \langle [\sigma \mid x : v]; v \rangle}$$

$$\frac{}{\langle \sigma; \mathsf{if\ true\ then}\ e_0\ \mathsf{else}\ e_1 \rangle \rightsquigarrow \langle \sigma; e_0 \rangle}$$

$$\frac{}{\langle \sigma; \mathsf{if\ false\ then}\ e_0\ \mathsf{else}\ e_1 \rangle \rightsquigarrow \langle \sigma; e_1 \rangle}$$

$$\frac{\langle \sigma; e \rangle \rightsquigarrow \langle \sigma'; e' \rangle}{\langle \sigma; \mathsf{if}\ e\ \mathsf{then}\ e_0\ \mathsf{else}\ e_1 \rangle \rightsquigarrow \langle \sigma'; \mathsf{if}\ e'\ \mathsf{then}\ e_0\ \mathsf{else}\ e_1 \rangle}$$

$$\frac{\langle \sigma; e_0 \rangle \rightsquigarrow \langle \sigma'; e_0' \rangle}{\langle \sigma; \mathsf{var}\ x = e_0\ \mathsf{in}\ e_1 \rangle \rightsquigarrow \langle \sigma'; \mathsf{var}\ x = e_0'\ \mathsf{in}\ e_1 \rangle}$$

$$\frac{x \notin \mathrm{dom}(\sigma)}{\langle \sigma; \mathsf{var}\ x = v_0\ \mathsf{in}\ e_1 \rangle \rightsquigarrow \langle [\sigma, x : v_0]; e_1 \rangle}$$

The answer should say something about using α-renaming to choose a fresh name that is not already in the domain of the store for the $\mathsf{var}\ x = v\ \mathsf{in}\ e$ case.

Type Systems

(*b*) (*i*) Define a reasonable set of types for this programming language. [1 mark]

(*ii*) Explain what a typing context should look like for this language. [1 mark]

(*iii*) Define a set of typing rules for this programming language, which should ensure type safety. [7 marks]

(*iv*) State (but do not prove) the progress and type preservation theorems for this language. [1 mark]

Answer:

(*i*) The relevant types are $T ::= \mathsf{bool} \mid \mathbb{N}$

(*ii*) Contexts are (a) partial maps from variables to types, OR (b) lists associating variables and types. Anything reasonable will do.

(*iii*) Some possible typing rules are:

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \qquad \frac{\Gamma \vdash e_0 : T \qquad \Gamma, x : T \vdash e_1 : T'}{\Gamma \vdash \mathsf{var}\ x = e_0\ \mathsf{in}\ e_1 : T'}$$

$$\frac{\Gamma \vdash e_0 : T \qquad x : T \in \Gamma}{\Gamma \vdash x := e_0 : T}$$

$$\frac{\Gamma \vdash e_0 : \mathbb{N} \qquad \Gamma \vdash e_1 : \mathbb{N}}{\Gamma \vdash e_0 + e_1 : \mathbb{N}} \qquad \frac{\Gamma \vdash e_0 : \mathbb{N} \qquad \Gamma \vdash e_1 : \mathbb{N}}{\Gamma \vdash e_0 < e_1 : \mathsf{bool}}$$

$$\frac{}{\Gamma \vdash \mathsf{true} : \mathsf{bool}} \qquad \frac{}{\Gamma \vdash \mathsf{false} : \mathsf{bool}}$$

$$\frac{\Gamma \vdash e : \mathsf{bool} \qquad \Gamma \vdash e_0 : T \qquad \Gamma \vdash e_1 : T}{\Gamma \vdash \mathsf{if}\ e\ \mathsf{then}\ e_0\ \mathsf{else}\ e_1 : T}$$

(*iv*) Before we can define type safety, we have to say what it means for a store σ to agree with an environment Γ.

(A) First, we define the $\mathsf{typeof} : \text{Value} \to \text{Type}$ function which return bool if it is handed true or false, and returns $\mathbb{N}$ if it is given a numeral n.

(B) Next, we say that σ *satisfies* Γ when for each variable x in the domain of Γ, $\mathsf{typeof}(\sigma(x)) = \Gamma(x)$.

- Progress: If $\Gamma \vdash e : T$ and σ satisfies Γ, then either e is a value or there exist σ and e' such that $\langle \sigma; e \rangle \rightsquigarrow \langle \sigma'; e' \rangle$.
- Type preservation: If $\Gamma \vdash e : T$ and σ satisfies Γ and $\langle \sigma; e \rangle \rightsquigarrow \langle \sigma'; e' \rangle$, then there is a $\Gamma' \supseteq \Gamma$ such that $\Gamma' \vdash e' : T$ and σ' satisfies Γ'.